Anatomy of the Intel Arc Battlemage GSC Firmware

Config-data verification, memory-geometry policy, and the B580 → 24 GB question

A research compilation — nine analysis sessions, October 2026

Abstract

Intel's Arc B580 (12 GB GDDR6) and B60 (24 GB GDDR6) share the BMG-G21 die and differ, at the firmware level, in a single signed 64 KB "config-data" blob. This paper documents a complete offline unpacking and analysis of the Battlemage GSC firmware (as shipped in the LVFS/fwupd update package `intel-arc-bmg-21.1180.cab`), including the first public, to our knowledge, full Huffman decompression of a BMG GSC code-partition image and a disassembly-level trace of the config-data deploy and boot paths.

We answer the question that gates any B580→24 GB memory modification: **who verifies the config-data signature, and when.** The answer is: effectively nobody, at firmware run time. The GSC `fwupdate` module accepts the 64 KB config blob with **structural checks only** (55AA/PCIR/market-manifest shape via `fn@0x6f7d`), writes it to the `bp1` flash partitions `OROM.man` / `GDTA.man`, and **no GSC module re-verifies it at boot** — the `rbe` boot ROM's manifest-name table contains no OROM/GDTA entries. Crypto enforcement of config-data lives entirely in the host-side tooling chain (igsc, LVFS jcat). The GSC's hard RSA-3072/PSS gate, anchored to fuse-derived key material via `/snowball/ipub_key`, protects **code manifests** (RBEP/PCOD/PHYP/OMSM) only.

We further show that (a) the B580↔B60 config diff is exactly 14 regions — one RSA-3072 signature, three SHA-384 hashes, a device-ID byte, a revision byte, and a handful of VBT display bytes — containing **no memory-geometry tables**; (b) the memory-training module (`fsp`) is **byte-identical** between the 12/24 GB G21 die and the 32 GB G31 die except for a message-ID table, so no per-die density cap exists at the GSC level; and (c) memory sizing is computed at run time from cached training results (`/snowball/mrc_data`, `/snowball/mrc_bdat`), not parsed from config content.

Consequence: the firmware-visible difference between a 12 GB and a 24 GB board is a signed identity token, and the binding constraint on any modification is the signing key at deploy time plus physical board wiring — not on-die verification code.

---

1. Artifacts and provenance

All firmware was obtained from public channels; chain of custody is clean.

Artifact	Size	Source	Role
`bmg_OpromCode.bin`	960 KB	LVFS cab	VBIOS/OpROM, all BMG SKUs
`bmg_g21_fwupdate.bin`	2.1 MB	LVFS cab	GSC firmware, BMG-G21 die (B580/B570/B50/B60)
`bmg_g31_fwupdate.bin`	2.1 MB	LVFS cab	GSC firmware, BMG-G31 die (B65/B70)
`bmg_e20b_1100_config19.bin`	64 KB	LVFS cab	config-data, **E20B = B580 (12 GB)**
`bmg_e212_1114_config23.bin`	64 KB	LVFS cab	config-data, **E212 = B60 (24 GB)**
`bmg_e223_c91_1701_config20.bin`	64 KB	LVFS cab	config-data, **E223 = B70**
`bmg_guc_70.bin` / `bmg_huc.bin`	386/586 KB	linux-firmware `xe/`	GuC/HuC microcode

The config-data blob `bmg_e20b_1100_config19.bin` was SHA-256 verified against the LVFS-published digest (`4103d75a…`).

2. Container structure

2.1 Flash image: Intel $FPT (IFWI) format

Both GSC update images open with magic `$FPT` (24 46 50 54) — the standard Intel flash partition table. Three partitions: `INFO` (2,560 B descriptor table), `IMGI` (@0xa00, `BMG_` tag with PCOD/p-code sections) and `FWIM` (@0x1000, the GSC firmware volume). Entropy analysis (~44 % of 4 KB blocks low-entropy) confirmed the container is parseable rather than fully encrypted.

2.2 Code Partition Directory: fully mapped

The `$CPD` directory at 0x2000 uses 24-byte entries `[type u32][name 8][pad u32][offset u32][size u32]` , offsets relative to the CPD base. 24 modules were catalogued; the compression flag is bit 25 (0x02000000) of the type word. Plain modules: `RBEP.man` , `rbe.met` , `intl.cfg` . Compressed: `rbe` , `kernel` , `syslib` , `pm` , `vfs` , `evtdisp` , `fpf` , `loadmgr` , `crypto` , `heci` , `storage` , `maestro` , `fwupdate` , `mca_boot` , `gfx_srv` , `fsp` , `sigma` , `cpassistant` , `mca_srv` .

2.3 Huffman decompression of all 20 modules (both dies) — solved

The compressed modules use CSME-layered Huffman compression. We built `tools/decode_gsc_cpd.py` around `csmeunpacker`'s `decode_page_from_input_file_i` and found the **csme12 Huffman table decodes BMG GSC cleanly** (csme11 yields noise). Correctness was arbitrated by strings: `/dev/maestro`, `../../src/os/servers/vfs/device.c`, `CSCP Kernel Memory Guard Key`, etc.

Two parser corrections mattered, and are recorded here because they will bite anyone reimplementing this:

1. **LUT selector mapping.** Entry top-2 bits `0xC0` → dict2 (selector 1); `0x40` → dict1 (selector 0). The little-endian dword "top bits" are *not* the selector value.
2. **LUT length is fixed**: exactly `size/4096` entries. Invalid entries (top bits neither 0x40/0xC0) are *skipped* (advance 4 bytes), not terminating. `lut_end = (size/4096)*4`; offsets are LUT-end-relative; page sizes = next valid entry's offset minus this offset; a last-page size of 0 means consume up to 4096 input bytes; decoded page size caps at 4096.

Every decoded page must target a real file-like object (`io.BytesIO`), not a bytearray.

Result: 20 Huffman modules decoded per die, output size equal to the CPD size field *exactly*, zero decode errors. G21 and G31 builds are near-identical (many modules byte-equal in size). All outputs are in `analysis/g21/` and `analysis/g31/`.

3. The config-data blob, dissected

3.1 Layout

The 64 KB config blob is plaintext (entropy 1.51) and begins with a `55AA` PCI expansion-ROM header:

Offset	Content
0x0000	`55AA` OROM header, PCIR
0x077E	device-ID byte (0x0B = E20B/B580; 0x12 = E212/B60)
0x0782	revision byte (0x00 vs 0x14 → config rev 1100 vs 1114)
0x0800	`GraphicsMem` OROM identifier (version string 2.5.0.1066 — *not* a size)
0x2440	`$VBT` `BATTLEMAGE` BIOS_DATA_BLOCK (LFP panels — display, not memory)
0x0520–0x069F	RSA-3072 signature

Offset	Content
0x06DC / 0x0730 / 0x079C	three SHA-384 hashes
0x06C0–0x07CC	tagged entry table: entries tagged `VBT`, `OROM`, device block

3.2 The B580 ↔ B60 diff: 538 of 65,536 bytes

A full byte diff of the E20B and E212 blobs yields 14 regions, and **only** these kinds of content: the RSA-3072 signature, the three SHA-384 hashes, the device-ID byte at 0x77E, the revision byte at 0x782, and ~9 bytes inside the VBT at 0x2440+ (display/panel blocks). **There are no memory-geometry or timing tables in the blob.** The 0x6C0–0x7CC region is a tagged entry table — entry 1 tag `VBT`, entry 2 tag `OROM` with a 0x100000 (1 MB) size field that exceeds the blob itself, entry 3 the device block (IDs 8086:E20B/1100 vs 8086:E212/1114) — each followed by one of the SHA-384s.

3.3 What the three SHA-384s hash

Exhaustive brute force (`tools/session7_analysis.py`): SHA-384/256 and truncated SHA-512 over every 0x40-aligned window of the blob, targeted unaligned starts/ends, crypto-zeroed variants, whole/sliced OpROM images, and HMAC variants keyed with the fuse-key names and the service GUID — **zero matches**. Because the `OROM` entry's size field (1 MB) exceeds the blob, the hashes evidently describe the *deployed flash sub-partitions* (the 1 MB OpROM/VBT/GDTA regions of bp1), computed at build time over content not shipped inside the blob. They are deploy-time cross-check material, not verified by anything in GSC firmware (§5–6).

3.4 Recovery variants

Each config ships a normal and a `_recovery` variant: identical content, separately signed (crypto regions and 4 version bytes differ). The partner-specific 258 KB config files in the LVFS package (Asrock/Sparkle/Shangke/IBC) are **all 0xFF placeholders**; only the three real configs (E20B, E212, E223) ship content.

4. The GSC code-signing chain

4.1 $MN2 manifests are CSE "R2" layout

The `RBEP.man` manifest matches MEAnalyzer's `MN2_Manifest_R2` struct: header length 0xE1 dwords (0x384 B), tag `$MN2` at 0x1C, RSA public key (96-dword modulus, little-endian) at 0x80–0x200, exponent at 0x200 = 0x00010001 (65537), signature at 0x204–0x384 (3072-bit). The signature scheme is **RSA-SSA-PSS**, not PKCS#1 v1.5.

The modulus is byte-identical in the G21 and G31 manifests — a genuine Intel GSC code-signing key (hex in `firmware/manifest_rsa3072_key.txt`).

4.2 The verify pipeline (fwupdate fn@0x3c84)

The disassembled manifest verifier parses the manifest, builds a per-module SHA-384 digest table, and hashes `manifest[0x00:0x80] + manifest[0x384:end]` — skipping the key and signature regions, confirming the R2 layout from the code itself — then does a 48-byte memcmp against the parent-manifest expected digest. The RSA-PSS operation runs in the imported syslib/crypto service (call-gate ioctl `fn@0x7229`). Our standalone PSS verification attempt against the embedded key *failed* (no valid PSS EM for any endianness pairing), implying either obfuscation of update-container manifests or that the verification key differs from the embedded one.

4.3 The anchor: fuse-derived keys, `/snowball/ipub_key`

No expected key-hash constant exists in any of the 40 decoded modules (re-scan confirmed). Instead, `rbe` (boot) keeps a key-store *name* table (`epid_keys`, `ftpm_fuses`, `curr_keys`, `const_keys`, `cse_gpk`, `rom_prk`, `LcWrapKey`, …) plus fused-key name strings: `'CSE Manifests Hash Key'`, `'CSE RBE Root Key'`, `'CSME ODCA Integrity Key'`, `'CSCP SVNx FW Root Key'`. These are resolved by the crypto service — **manifest hash/MAC keys are derived on device from eFuse/FPF material.**

The trust anchor data structure is `/snowball/ipub_key`: `fn@rbe 0x4511` reads a header with slot count (≤7) and validity mask, then copies 48 bytes (SHA-384) per slot — the trusted store is a list of up to 8 public-key hashes. `PreOs_VerifyHash` (`fn@rbe 0x42a8`) builds a 0x3D0-byte descriptor and memcmps 48 bytes against the expected value. `loadmgr` independently consumes `/snowball/ipub_key` + `/snowball/oem_key_man` for loaded-module verification.

Chain: a manifest's embedded RSA-3072 key is trusted iff its SHA-384 is one of the `ipub_key` slots; `ipub_key` itself is authenticated by the fuse-anchored chain.

5. Who verifies config-data, and when: deploy-time only

This is the paper's central result. Three independent lines of evidence:

(1) The update dispatcher routes OROM/GDTA away from the crypto path. `fwupdate`'s HECI state machine (`fn@0x14d2`, states 1–5) routes by partition type: types 3/4 → buffered path → `$MN2` verify (`fn@0x3c84`) → SHA/RSA-PSS. **Types 18/19 (0x12/0x13) = OROM.man/GDTA.man** take a per-0x1000-chunk flash-write path

(`fn@0x5b85` , import 0xC0001421) whose last-chunk branch calls `fn@0x56f0` — which reads the `GDTA.man` / `OROM.man` / `FspInfo` flash entries (find-by-name `fn@0x6ecb/0x6e34` , region IDs 0x1047/0x1048, size cap 0x46000) and calls only the **structural verifier `fn@0x6f7d` **. Call-graph analysis (`tools/session7_analysis.py`) proves the transitive callees of `fn@0x56f0` contain *neither* the `$MN2` verifier *nor* the RSA-PSS ioctl: the two paths share only the dispatcher ancestor.

(2) No offset-level or device-ID enforcement anywhere. Exhaustive immediate scans of all 20 modules (both dies) found: no config-blob offsets (0x520/0x6A0/0x6DC/0x730/0x79C/0x77E/0x782) used as comparison targets; no PCI-ID immediates 0xE20B/0xE212/0xE223 in any decoded module; no anti-rollback comparison of the revision byte. Device acceptance is implicit in the signing chain; CPU/die fields are logged, never compared.

(3) Nothing at boot touches it. `rbe` 's manifest-name table lists only `OMSM/PCOD/PHYP/rbe/oem_key_man/intl.cfg/soc_cfg_lok/feature_map` . The strings `GDTA` / `OROM.man` / `GDTA.man` appear **only in fwupdate** across all 40 modules. `mca_boot` reads `/dev/cfg_files` + `/susram/GfxDtaUpdSts` — update *status* only.

Verdict: the 64 KB config blob is deployed with structural checks only, written to bp1, and never re-verified. Crypto enforcement of config-data lives in the host tooling chain (igsc / LVFS jcat). The mystery 16-byte constant at fwupdate 0x7F48 resolved to the fwupdate **service registration GUID** (`870d0ca5-3495-4559-8105-3fbfa37b8b79` , mixed-endian) — not a key hash.

6. Runtime consumers: none derive memory size from config content

A full sweep of the remaining modules (`gfx_srv` , `fsp` , `kernel` , `pm` , `mca_srv` , `cpassist`) found **no on-die code that reads or derives memory geometry from config-data**:

- `gfx_srv` (24 KB) is a thin status/token/mailbox shim (FSP handshake, ELG Token 1.0, `/dev/flash/region12` status page) — zero GDTA/GfxDta/VBT references.
- `fsp` (122 KB) consumes **memory-training data**: `fn@0xC0023674` stages both NVM vars `/snowball/mrc_data` → SRAM 0x10031000 and `/snowball/mrc_bdat` → SRAM 0x10007000, 0x7F80 B each. It contains real MMIO-level PHY init (read-modify-write helpers writing regs 0x8828, 0x8800, 0x4910, 0xA180/0xA194 over the 0x0010_xxxx PHY window) — memory size is programmed dynamically (64-bit `shrd 0x11` / `shl 0xE` into reg 0x8800), never as a hardcoded constant.
- `kernel` / `pm` / `mca_srv` / `cpassist` : no config or geometry references.

VA mapping note

All Huffman modules link at the **same** link-time base: `VA = 0xC0000000 + 0x22000 + file_off` uniformly (verified against 32/34 gfx_srv string xrefs and every fwupdate `.man` string). The session-7 per-module "bases" from bootstrap immediates are per-module *data-segment* anchors, not load bases. Cross-module negative rel32 targets land in a compact call-gate/import page at 0xC0001000–0xC0002000.

7. No density cap at the GSC level

`fsp` is **byte-identical** between the G21 (12/24 GB) and G31 (32 GB B70) dies except for one region, 0x1BDE0–0x1C2B4 — a (message-ID, event-type) table where every G31 ID is the G21 ID + 0x10. Zero code bytes differ. The identical trainer therefore runs on the die that ships 32 GB; a G21-side 16 Gbit-per-chip cap *cannot* be absent from G21 `fsp` while present in G31 `fsp`. No `(density, rank, speed)` lookup table exists in code immediates; the only 0x800000 immediates are SRAM buffer sizes.

Residual unknowns: (a) the actual MRC training engine may live in the SRAM-staged `mrc_data` blob itself, whose contents are not in these images; (b) PCOD/pcode was not examined; (c) the real-world gates remain the signed config blob at deploy time, board straps/wiring, and BIOS/VBT.

8. Implications

1. **The B580→24 GB firmware gate is a signing key, not a check.** Swapping a config blob requires one that passes the host-side igsc check and the structural deploy path — GSC firmware itself would boot an unsigned/tampered flash partition. But a *valid* B60 blob differs from the B580 blob only by device ID + signature, and obtaining (or forging) a signature for modified content is equivalent to holding Intel's config-signing private key.
2. **Tampering route that bypasses all firmware crypto:** direct SPI flash writes of the GDTA/OROM partitions boot unverified (confirmed: `rbe` never references them). What stops this in practice is physical access and whether host-side components (BIOS/OpROM consumption of VBT and the device block) tolerate the change.
3. **Memory size policy is not in config-data.** 12 vs 24 GB firmware-wise reduces to the signed device-ID/manifest identity; actual sizing is determined by memory training and board wiring.

4. **The GSC is not a black box.** The directory layer is 100 % mapped and every module decompressed; the tooling in `tools/` reproduces all of it from the public LVFS package.

9. Open questions

1. RSA-PSS verify inputs in syslib/crypto (the call-gate ioctl surface at `fn@0x7229`).
2. Why the standalone PSS verify against the embedded key fails — obfuscated update manifests vs. a separate config key.
3. Definitive mapping of the blob's 3 SHA-384s to bp1 sub-regions (VBT/CODE/GDTA tags at 0x7EF8–0x7F1D in fwupdate data).
4. Contents of the `mrc_data` training engine blob.
5. Whether host-side igsc enforces the device-ID byte at 0x77E.
6. PCOD/pcode side of the config story.

10. Reproducing

See `../README.md` for the exact pipeline. In short:
`tools/decode_gsc_cpd.py` unpacks the $FPT/$CPD/Huffman layers into `analysis/g21|g31/`; `tools/parse_manifest.py` decodes `$MN2` R2 manifests; `tools/session7_analysis.py` (bases, call graph, hash brute), `tools/session8_sweep.py` / `session8_xref.py` (string xrefs, immediate scans) and `tools/session9_*.py` (fsp diff, callgraphs, xrefs) reproduce every claim in §4–7 from the decoded modules and the config blobs in `firmware/config-data/`.

Artifacts: LVFS cab `intel-arc-bmg-21.1180.cab` (fwupd, cdn.fwupd.org) and linux-firmware `xe/` tree. All analysis performed offline, no hardware attached.